

Aula 08 — Spring Framework

Thymeleaf · Banco de dados · Projeto banco

Mauricio Duailibi Neto

Turma Java Entra21

Esta aula tem dois dias

Dia 1 · Parte 1 — 18h30 às 20h

- O que é Thymeleaf
- O que é banco de dados
- Instalar o XAMPP

Dia 2 · Parte 3 — 18h30 às 20h

- Login com CPF e senha
- Página da conta com o saldo
- Depósito e saque

Dia 1 · Parte 2 — 20h20 às 22h

- O projeto banco
- Configurar o banco no Spring
- Tela de cadastro gravando no MySQL

Dia 2 · Parte 4 — 20h20 às 22h

- PIX para outra conta
- Extrato
- Teste com pessoas do gerador

Duas novidades

Até a aula 07 a gente fazia sempre do mesmo jeito.
A partir de hoje, duas coisas mudam.

Thymeleaf O Java passa a montar a página HTML.
Sem Axios. Sem `app.js`.

Banco de dados Os dados deixam de ficar no `ArrayList`.
Vão para o MySQL e não somem quando o Spring para.

O projeto que junta as duas coisas é um **banco**:
conta, saldo, depósito, saque e PIX.

Antes de começar: baixem o XAMPP

O download demora um pouco.

Deixem baixando enquanto a gente conversa.

- 1 Abram <https://www.apachefriends.org>
- 2 Cliquem em **XAMPP para Windows**
- 3 Só baixem. **Não instalem ainda.**

Daqui a pouco a gente instala junto, passo a passo.

Mapa da aula

- 1 Como fazíamos até agora
- 2 O que é Thymeleaf
- 3 A sintaxe do Thymeleaf
- 4 Banco de dados
- 5 XAMPP e phpMyAdmin
- 6 JPA: o Java fala com o banco
- 7 O projeto banco
- 8 Configurando o projeto

- 9 Cadastro de contas
- 10 Dia 2 — Login e sessão
- 11 A página da conta
- 12 Depósito e saque
- 13 PIX
- 14 Extrato
- 15 Teste final
- 16 Recapitulando

Relembrando o ATM da aula 07

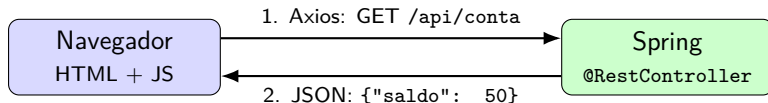
No caixa eletrônico da aula passada:

- `index.html` ficava na pasta `static`
- `app.js` usava o `Axios` para chamar `/api/conta`
- O Java respondia com JSON: `{"saldo": 50}`
- O JavaScript pegava o número e colocava na tela

O Java só mandava **dados**.

Quem montava a tela era o JavaScript, no navegador.

O desenho de antes



3. O JavaScript pega o 50 e escreve R\$ 50,00 na tela.

Isso se chama **API REST**.

Foi o que fizemos da aula 01 até a aula 07.

O problema da memória

Onde ficavam os alunos, os produtos, as vendas?

Num ArrayList, dentro da classe Dados.

O que acontece quando o Spring para

Tudo some.

Cadastrou dez produtos, parou o Spring, subiu de novo:
a lista volta vazia.

Num banco de verdade isso não pode acontecer.

Ninguém aceita perder o saldo porque o servidor reiniciou.

Uma comparação com restaurante

Antes (API REST)

A cozinha manda os ingredientes separados para a mesa:
o arroz, o feijão, a carne.
Quem monta o prato é o cliente (o JavaScript).

Agora (Thymeleaf)

A cozinha já manda o **prato montado**.
O cliente só come.

O “prato montado” é a página HTML com os dados já dentro.

O que é o Thymeleaf

É uma ferramenta que o Spring usa para montar páginas HTML.

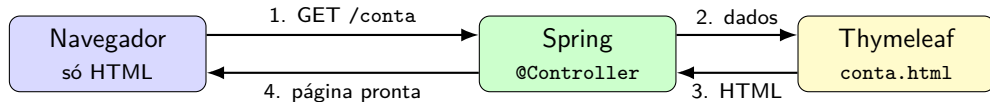
A gente escreve um HTML normal,
com alguns espaços marcados para o Java preencher.

Antes de mandar a página para o navegador,
o Thymeleaf troca esses espaços pelos dados de verdade.

Em uma frase

O servidor devolve a página pronta, já com o nome e o saldo.

O desenho de agora



O navegador recebe `<p>R$ 50,00</p>` já escrito.
Não tem Axios. Não tem JavaScript para montar a tela.

Um exemplo pequeno

No arquivo HTML a gente escreve:

```
<p th:text="${nome}">aqui vai o nome</p>
```

No Java a gente diz qual é o nome:

```
model.addAttribute("nome", "Ana");
```

O navegador recebe:

```
<p>Ana</p>
```

O `th:text` trocou o texto de dentro pelo valor do Java.

Antes e agora, lado a lado

	Antes (REST)	Agora (Thymeleaf)
Anotação	@RestController	@Controller
O Java devolve	dados (JSON)	o nome da página
Quem monta a tela	JavaScript	o servidor
Como o form envia	Axios	o próprio <form>
Pasta do HTML	static/	templates/
Arquivo app.js	sim	não precisa

A pasta `static/` continua existindo.
Ela guarda o CSS e as imagens.

A diferença no Controller

Antes

```
@RestController
public class ContaController {

    @GetMapping("/api/conta")
    public Conta ver() {
        return conta;
    }
}
```

Devolve o objeto.
Vira JSON.

Agora

```
@Controller
public class ContaController {

    @GetMapping("/conta")
    public String ver(Model model) {
        model.addAttribute("conta", conta);
        return "conta";
    }
}
```

Devolve o texto "conta".
O Spring abre templates/conta.html.

Qual dos dois é melhor?

Nenhum. São usados em situações diferentes.

REST Quando a tela é separada do servidor:
aplicativo de celular, site feito em React, outro sistema.

Thymeleaf Quando o site inteiro fica num projeto só.
Muito sistema interno de empresa é assim.

Os dois são Spring.

Quem sabe os dois consegue trabalhar em qualquer um.

A primeira linha muda

Todo arquivo que usa Thymeleaf começa assim:

```
<!DOCTYPE html>  
<html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
```

O `xmlns:th` avisa: “neste arquivo existem comandos `th:`”.

Sem essa linha a página até funciona,
mas o VS Code sublinha tudo que começa com `th:`.

O Model é uma bandeja

O Controller coloca os dados numa bandeja.

A página pega os dados da bandeja.

```
@GetMapping("/conta")  
public String ver(Model model) {  
    model.addAttribute("nome", "Ana");  
    model.addAttribute("saldo", 250.0);  
    return "conta";  
}
```

"nome" é a etiqueta.

"Ana" é o que está dentro.

Na página, a gente chama pela etiqueta: \${nome}.

th:text — escrever um valor

```
<p th:text="${nome}">Nome</p>
<p th:text="'Saldo: R$ ' + ${saldo}">R$ 0</p>
```

- `${...}` busca no Model
- O texto entre as tags (Nome) é trocado
- Texto fixo vai entre aspas simples: `'Saldo: R$ '`
- O `+` junta os pedaços, igual no Java

Se o objeto tiver atributos, a gente usa o ponto:
`${conta.nome}` chama o `getNome()` da conta.

th:if — mostrar só às vezes

```
<p class="alerta-erro" th:if="${erro}" th:text="${erro}"></p>
```

Se existir um erro no Model, o parágrafo aparece.

Se não existir, o parágrafo nem vai para o navegador.

É o `if` do Java, dentro do HTML.

Vamos usar para as mensagens:

“Saldo insuficiente”, “CPF ou senha incorretos”.

th:each — repetir para cada item

```
<ul>  
  <li th:each="nome : ${nomes}" th:text="${nome}"></li>  
</ul>
```

Se nomes for uma lista com Ana, Bruno e Carla, sai:

```
<ul>  
  <li>Ana</li>  
  <li>Bruno</li>  
  <li>Carla</li>  
</ul>
```

É o for do Java.

Vamos usar no extrato: uma linha para cada movimentação.

th:action e th:href — os endereços

```
<form th:action="@{/deposito}" method="post">  
<a th:href="@{/sair}">Sair</a>
```

`${...}` é para **dados**.

`@{...}` é para **endereços**.

O `th:action` diz para onde o formulário vai.

O `th:href` diz para onde o link vai.

Funcionaria com `action` e `href` comuns também.

Usamos a versão `th`: porque é o jeito do Thymeleaf.

O formulário volta a trabalhar sozinho

Sem Axios, quem envia é o próprio <form>.

```
<form th:action="@{/deposito}" method="post">
  <input type="number" name="valor">
  <button type="submit">Depositar</button>
</form>
```

No Java, o name do input vira o parâmetro:

```
@PostMapping("/deposito")
public String depositar(@RequestParam double valor) {
```

name="valor" no HTML, valor no Java.

Se os nomes forem diferentes, dá erro.

redirect — mandar o navegador para outro lugar

```
return "conta";           // mostra templates/conta.html  
return "redirect:/conta"; // manda o navegador abrir /conta
```

Depois de um POST, a gente sempre usa redirect.

Por quê?

Sem o redirect, se a pessoa apertar F5 depois de depositar, o navegador manda o depósito de novo.

Com o redirect, o F5 só recarrega a página da conta.

O navegador nunca vê o th:

Cliquem com o botão direito na página e em **Exibir código-fonte**.

Não aparece `th:text`, nem `${conta.nome}`.

Aparece o HTML pronto, com o nome escrito.

Todo o trabalho do Thymeleaf acontece no servidor, antes da página sair.

Comando	Para que serve	Parecido com
<code>th:text</code>	escrever um valor	<code>System.out.println</code>
<code>th:if</code>	mostrar só se for verdade	<code>if</code>
<code>th:each</code>	repetir para cada item	<code>for</code>
<code>th:action</code>	para onde o form vai	URL do Axios
<code>th:href</code>	para onde o link vai	<code>href</code>
<code>\${...}</code>	pegar dado do Model	variável
<code>@{...}</code>	montar um endereço	<code>"/api/..."</code>

É só isso que vamos usar nessa aula.

O que é um banco de dados

Um programa que guarda informação no disco, de forma organizada.

Pensem numa planilha do Excel:

Excel	Banco de dados
O arquivo	o banco
Cada aba	uma tabela
Cada coluna	uma coluna (nome, cpf, saldo)
Cada linha	um registro (uma pessoa)

Desligou o computador? Os dados continuam lá.

Uma tabela de verdade

A tabela conta, que vamos criar hoje:

id	nome	cpf	senha	saldo
1	Ana Souza	11111111111	123	250.25
2	Bruno Lima	22222222222	456	150.25
3	Carla Dias	33333333333	789	0.00

Cada linha é uma conta.

É igual ao `ArrayList<Conta>`, só que guardado no disco.

A chave primária

A coluna `id` é especial.

- Nunca se repete
- Nunca fica vazia
- O próprio banco escolhe o número: 1, 2, 3...

Ela se chama **chave primária**.

Lembram do `Dados.proximoId = Dados.proximoId + 1`?

O banco faz isso sozinho.

MySQL Um dos bancos de dados mais usados no mundo.
É gratuito.

SQL A língua que se usa para falar com o banco.
Serve para o MySQL e para quase todos os outros.

O SQL tem quatro comandos principais.
Vocês já conhecem a ideia.

Os quatro comandos

SQL	O que faz	Parecido com
SELECT	ler	GET
INSERT	criar	POST
UPDATE	alterar	PUT
DELETE	apagar	DELETE

É o mesmo CRUD das aulas 03 e 04.
Só que agora falando direto com o banco.

Como o SQL se parece

```
SELECT * FROM conta;
```

```
SELECT nome, saldo FROM conta WHERE cpf = '11111111111';
```

```
INSERT INTO conta (nome, cpf, senha, saldo)  
VALUES ('Ana Souza', '11111111111', '123', 0);
```

```
UPDATE conta SET saldo = 500 WHERE id = 1;
```

Dá para ler quase como português:

“selecione nome e saldo da conta onde o cpf é...”

Vamos precisar escrever SQL?

Quase nada.

O Spring escreve o SQL por nós.

A gente chama um método Java e ele monta o comando.

Vamos usar o SQL só para **olhar** o banco:
conferir se a conta foi gravada, se o saldo mudou.

Para isso existe o phpMyAdmin.

O que é o XAMPP

Um instalador que traz várias ferramentas juntas.

Das que vêm nele, vamos usar três:

MySQL O banco de dados.

(O XAMPP traz o MariaDB, que é o MySQL com outro nome.)

phpMyAdmin Uma página para ver e mexer no banco pelo navegador.

Apache Só serve para abrir o phpMyAdmin.

O nosso Spring **não** usa o Apache.

O Spring conversa direto com o MySQL.

Passo 1 — Instalar o XAMPP

- 1 Abram o arquivo que baixaram
- 2 Se o Windows perguntar se permite alterações: **Sim**
- 3 Se aparecer um aviso sobre antivírus ou UAC: **OK**
- 4 Nos componentes, deixem como está e cliquem em **Next**
- 5 Pasta: deixem C:\xampp
- 6 **Next** até terminar

Firewall

Se o Windows perguntar sobre o firewall, cliquem em **Permitir**.

Passo 2 — Ligar o MySQL

Abram o **XAMPP Control Panel**.

- 1 Na linha **Apache**, cliquem em **Start**
- 2 Na linha **MySQL**, cliquem em **Start**
- 3 Os dois nomes ficam com fundo verde
- 4 Na linha do MySQL aparece a porta 3306

Todo dia de aula

O MySQL precisa estar ligado **antes** de subir o Spring.
Sem ele, o Spring não consegue iniciar.

Passo 3 — Abrir o phpMyAdmin

No navegador:

`http://localhost/phpmyadmin`

Do lado esquerdo aparece a lista de bancos que já existem.

Alguns vêm com o XAMPP: `mysql`, `phpmyadmin`, `test`.

Não mexam neles.

Vamos criar o nosso.

Passo 4 — Criar o banco

- 1 No lado esquerdo, cliquem em **Novo** (ou **New**)
- 2 Nome do banco: banco
- 3 Ao lado, escolham `utf8mb4_general_ci`
- 4 Cliquem em **Criar** (ou **Create**)

O banco banco aparece na lista da esquerda.

Ainda sem nenhuma tabela.

O `utf8mb4` deixa gravar acento e cedilha sem virar símbolo estranho.

Outro jeito: pela aba SQL

Quem preferir, pode criar escrevendo o comando.

Cliquem na aba **SQL**, colem e cliquem em **Executar**:

```
CREATE DATABASE banco
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;
```

O resultado é o mesmo do passo 4.

Não precisa fazer os dois.

Usuário e senha do banco

O MySQL do XAMPP vem com:

- usuário: root
- senha: nenhuma, em branco

Isso só serve no nosso computador, para estudar.

Um banco de verdade, na internet, sempre tem senha forte.

Vamos colocar esse usuário na configuração do Spring daqui a pouco.

Se o XAMPP não ligar

- MySQL não fica verde — outro MySQL já está instalado no computador usando a porta 3306. Chamem o professor.
- Apache não fica verde — a porta 80 está ocupada (Skype, IIS). O phpMyAdmin não abre, mas o Spring funciona.
- MySQL shutdown unexpectedly — chamem o professor.
- phpMyAdmin não abre — confirmam se o Apache está verde.

Checkpoint

XAMPP instalado

Apache e MySQL verdes no painel

`http://localhost/phpmyadmin` abre

O banco banco aparece na lista da esquerda

Todo mundo com os quatro itens?

Então vamos ver como o Java fala com esse banco.

Duas línguas diferentes

O Java pensa em **objetos**: uma Conta com nome e saldo.

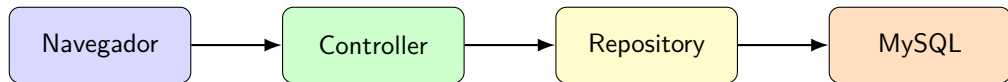
O banco pensa em **tabelas**: linhas e colunas.

Alguém precisa traduzir.

Esse tradutor se chama **JPA**.

Java	Banco
classe Conta	tabela conta
atributo saldo	coluna saldo
um objeto Conta	uma linha da tabela

Quem conversa com quem



- O **Controller** recebe o pedido da tela
- O **Repository** é quem fala com o banco
- O Controller nunca escreve SQL. Ele pede para o Repository.

Anotações na classe

```
@Entity
public class Conta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String cpf;
```

- @Entity — esta classe vira uma tabela
- @Id — este atributo é a chave primária
- @GeneratedValue — o banco escolhe o número
- @Column(unique = true) — não pode repetir

Por que Long e não int?

Até agora o id era `int`.

Com banco de dados a gente usa `Long`.

- `Long` é um número inteiro, que cabe valores bem grandes
- Com `L` maiúsculo ele pode ficar **vazio** (`null`)
- Antes de salvar, a conta ainda não tem id: fica `null`
- Depois de salvar, o banco preenche: 1, 2, 3...

É o costume em todo projeto Spring com banco.

O Repository

```
public interface ContaRepository  
    extends JpaRepository<Conta, Long> {  
}
```

É uma **interface**, não uma classe.

A gente não escreve o código dos métodos.

O Spring lê <Conta, Long> e entende:

“é um repositório de Conta, e o id é Long”.

Ao subir, o Spring cria sozinho a classe que faz o trabalho.

Métodos que já vêm prontos

Antes, com Dados	Agora, com o Repository
<code>Dados.contas.add(conta)</code>	<code>contaRepository.save(conta)</code>
<code>Dados.contas</code> for procurando o id	<code>contaRepository.findAll()</code> <code>contaRepository.findById(id)</code>
<code>Dados.contas.remove(...)</code>	<code>contaRepository.deleteById(id)</code>

O save serve para os dois casos:
se a conta é nova, faz INSERT.
Se já existe, faz UPDATE.

Um método que a gente inventa

Para o login, precisamos achar a conta pelo CPF.

```
Conta findByCpf(String cpf);
```

Só escrevemos essa linha, sem corpo.

O Spring lê o **nome** do método:

Pedaço	Significa
find	buscar
By	onde
Cpf	o atributo cpf é igual ao que eu passar

E monta: `SELECT * FROM conta WHERE cpf = ?`

Se não achar, devolve null.

@Autowired — pedir um pronto

```
@Controller
public class AcessoController {

    @Autowired
    private ContaRepository contaRepository;
```

A gente não faz `new ContaRepository()`.

Nem daria: é uma interface.

O `@Autowired` diz ao Spring:

“me entrega aquele repositório que você criou”.

E a tabela, quem cria?

Também o Spring.

Na configuração vamos colocar:

```
spring.jpa.hibernate.ddl-auto=update
```

Quando o Spring sobe, ele olha as classes com @Entity:

- Se a tabela não existe, ele cria
- Se faltar uma coluna, ele acrescenta
- Os dados que já estão lá, ele não apaga

A gente só cria o **banco** no phpMyAdmin.

As **tabelas** ficam por conta do Spring.

- 1 Thymeleaf: o servidor devolve a página pronta
- 2 @Controller devolve o nome do HTML, que fica em templates/
- 3 O Model leva os dados para a página
- 4 Banco de dados guarda no disco. Nada some ao reiniciar.
- 5 @Entity vira tabela, o Repository fala com o banco
- 6 O XAMPP liga o MySQL. O phpMyAdmin mostra o que tem dentro.

Retomamos às **20h20**

Deixem o XAMPP aberto, com o MySQL ligado.
Na volta a gente abre o projeto **banco**.

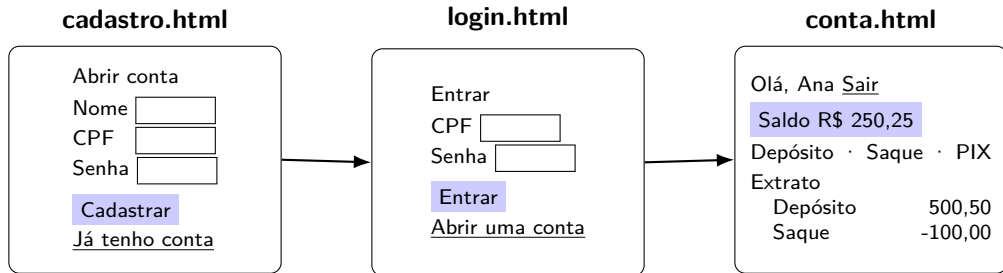
O que vamos construir

O **Banco Entra21**. Nele, a pessoa pode:

- 1 Abrir uma conta com nome, CPF e senha
- 2 Entrar com CPF e senha
- 3 Ver o próprio saldo
- 4 Depositar
- 5 Sacar, se tiver saldo
- 6 Mandar um PIX para outra conta, usando o CPF
- 7 Ver o extrato de tudo que aconteceu

É o ATM da aula 07, só que com várias contas e com o dinheiro guardado no banco de dados.

As três telas



O caminho de quem usa

- 1 Abre /cadastro, preenche e clica em Cadastrar
- 2 O Java grava a conta no banco, com saldo zero
- 3 Vai para /login, digita CPF e senha
- 4 O Java confere no banco e guarda quem entrou
- 5 Vai para /conta: nome, saldo e as operações
- 6 Cada operação é um formulário que volta para /conta
- 7 /sair esquece quem estava logado

Todos os endereços

Verbo e endereço	O que manda	O que acontece
GET /cadastro	—	mostra a tela de cadastro
POST /cadastro	nome, cpf, senha	grava a conta
GET /login	—	mostra a tela de login
POST /login	cpf, senha	confere e entra
GET /conta	—	mostra saldo e extrato
POST /deposito	valor	soma no saldo
POST /saque	valor	tira do saldo
POST /pix	cpfDestino, valor	passa para outra conta
GET /sair	—	sai da conta

Nenhum endereço começa com /api.

Todos devolvem página, não JSON.

As duas tabelas

conta

- id
- nome
- cpf
- senha
- saldo

movimentacao

- id
- descricao — “Depósito”, “PIX enviado para Bruno”
- valor — negativo quando sai dinheiro
- data_hora
- conta_id — de quem é

Uma conta tem várias movimentações.

É o 1:N da aula 05: o `conta_id` faz o papel do `categoriaId`.

Sessão: o crachá

Depois do login, como o Spring sabe quem está usando?

Pensem num prédio com portaria:

- Na portaria, você mostra o documento (**login**)
- Recebe um crachá (**sessão**)
- Em cada andar, o segurança olha o crachá (**/conta**)
- Na saída, devolve o crachá (**/sair**)

No Java o crachá é o `HttpSession`.

Nele guardamos o id da conta que entrou.

As classes do projeto

Pasta	Arquivo	Para que serve
model	Conta	a tabela conta
model	Movimentacao	a tabela movimentacao
repository	ContaRepository	fala com a tabela conta
repository	MovimentacaoRepository	fala com a movimentacao
controller	AcessoController	cadastro, login, sair
controller	ContaController	saldo e operações
templates	3 arquivos .html	as três telas

O CSS já está pronto. O resto a gente escreve junto.

O que fica para cada dia

Quando	O que fica pronto
Hoje, depois do intervalo	configuração, Conta, cadastro
Amanhã, primeira parte	login, página da conta, depósito, saque
Amanhã, segunda parte	PIX, Movimentacao, extrato

Ao fim de hoje, cada cadastro já aparece no phpMyAdmin.

Passo 1 — Abrir o projeto

Onde estamos

Projeto **banco**

- 1 Atualizem o repositório (`git pull`)
- 2 No VS Code: **File > Open Folder**
- 3 Escolham a pasta banco

O projeto foi gerado no `start.spring.io` com:

- Spring Web
- Thymeleaf
- Spring Data JPA
- MySQL Driver

O que já veio pronto

```
banco/  
  pom.xml  
  src/main/java/com/entra21/banco/  
    BancoApplication.java  
  src/main/resources/  
    application.properties  
    static/css/estilo.css
```

Só isso. Nenhum Controller, nenhum HTML.

Não subam o Spring ainda

Com o JPA no projeto, o Spring quer um banco configurado.
Se subir agora, dá erro. Primeiro o passo 2.

Onde cada arquivo vai ficar

```
banco/  
  src/main/java/com/entra21/banco/  
    BancoApplication.java  
    model/Conta.java             hoje  
    model/Movimentacao.java      amanha  
    repository/ContaRepository.java hoje  
    repository/MovimentacaoRepository.java amanha  
    controller/AcessoController.java hoje e amanha  
    controller/ContaController.java amanha  
  src/main/resources/  
    application.properties       hoje  
    static/css/estilo.css        pronto  
    templates/cadastro.html      hoje  
    templates/login.html         amanha  
    templates/conta.html         amanha
```

Passo 2 — Configurar o banco

Onde estamos

src/main/resources/application.properties

```
spring.application.name=banco

spring.datasource.url=jdbc:mysql://localhost:3306/banco
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

server.servlet.session.tracking-modes=cookie
```

A primeira linha já estava lá. Acrescentem o resto.

O que cada linha faz

`datasource.url` Onde está o banco.

`localhost:3306` é o MySQL do XAMPP. `/banco` é o nome que criamos.

`username e password` root, sem senha.

`ddl-auto=update` Criar e atualizar as tabelas sozinho.

`show-sql=true` Mostrar no terminal o SQL que o Spring escreve.

`tracking-modes` Guardar o crachá só no navegador,
sem aparecer um código estranho no endereço.

Passo 3 — A classe Conta

Onde estamos

Pasta nova model, arquivo Conta.java

```
@Entity
public class Conta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;

    @Column(unique = true)
    private String cpf;

    private String senha;

    private double saldo;
```

Passo 3 — Completar a Conta

Onde estamos

Ainda em `model/Conta.java`

- Construtor vazio: `public Conta() { }`
- Gets e sets dos cinco atributos

Os imports vêm de `jakarta.persistence`:

`Entity, Id, GeneratedValue, GenerationType, Column.`

Cuidado no import

Se o VS Code oferecer `javax.persistence`, não aceitem.
O certo é `jakarta.persistence`.

Passo 4 — Subir o Spring

Confirmam: o MySQL está verde no XAMPP?

No terminal, dentro da pasta banco:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Na primeira vez demora: o Maven baixa o JPA e o Thymeleaf.

Um Spring por vez

Se outro projeto ainda estiver na porta 8080, parem o outro antes.

O que aparece no terminal

No meio das mensagens, procurem por esta:

```
Hibernate: create table conta (id bigint not null auto_increment,  
    cpf varchar(255), nome varchar(255), saldo float(53) not null,  
    senha varchar(255), primary key (id))
```

É o SQL que o Spring escreveu a partir da classe Conta.

A gente não escreveu nenhuma linha dele.

Aparece por causa do `show-sql=true`.

Passo 5 — Ver a tabela no phpMyAdmin

- 1 Voltem para `http://localhost/phpmyadmin`
- 2 Cliquem no banco banco, na esquerda
- 3 Apareceu a tabela conta
- 4 Cliquem nela e depois em **Estrutura**

Estão lá as cinco colunas: id, cpf, nome, saldo, senha.

A tabela ainda está vazia.

Quem vai preencher é a tela de cadastro.

Passo 6 — O ContaRepository

Onde estamos

Pasta nova repository, arquivo ContaRepository.java

```
public interface ContaRepository
    extends JpaRepository<Conta, Long> {

    Conta findByCpf(String cpf);
}
```

Atenção: é interface, não class.

Imports: JpaRepository e Conta.

Onde fica o HTML agora

Os HTML do Thymeleaf ficam numa pasta nova:

```
src/main/resources/templates/
```

Criem essa pasta, ao lado da `static`.

Não confundam

`static/` — arquivos que vão do jeito que estão (CSS)

`templates/` — páginas que o Thymeleaf preenche antes

Passo 7 — cadastro.html, o topo

Onde estamos

Arquivo novo templates/cadastro.html

```
<!DOCTYPE html>
<html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Abrir conta</title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <div class="cartao">
    <p class="marca">Banco Entra21</p>
    <h1 class="titulo">Abrir conta</h1>

    <p class="alerta-sucesso" th:if="${sucesso}" th:text="${sucesso}"></p>
    <p class="alerta-erro" th:if="${erro}" th:text="${erro}"></p>
```

O arquivo continua no próximo slide.

Passo 7 — cadastro.html, o formulário

Onde estamos

Continuação de templates/cadastro.html

```
<form th:action="@{/cadastro}" method="post">
  <label>Nome completo</label>
  <input type="text" name="nome" required>

  <label>CPF (só números)</label>
  <input type="text" name="cpf" maxlength="11" required>

  <label>Senha</label>
  <input type="password" name="senha" required>

  <button class="botao" type="submit">Cadastrar</button>
</form>

<a class="link" th:href="@{/login}">Já tenho conta</a>
</div>
</body>
</html>
```

- As classes (cartao, botao...) já estão no CSS
- Os dois `th:if` mostram mensagem só quando existe
- O `th:action` manda o formulário para POST /cadastro
- Os `name` são nome, cpf e senha:
iguais aos atributos da classe Conta
- `maxlength="11"`: o CPF só com números tem 11 dígitos
- `required`: o navegador não deixa enviar vazio

Passo 8 — AcessoController, mostrar a tela

Onde estamos

Pasta nova controller, arquivo AcessoController.java

```
@Controller
public class AcessoController {

    @Autowired
    private ContaRepository contaRepository;

    @GetMapping("/cadastro")
    public String telaCadastro() {
        return "cadastro";
    }
}
```

@Controller, **não** @RestController.

O "cadastro" é o nome do arquivo, sem o .html.

Testando a tela

Parem o Spring (Ctrl+C) e subam de novo.

Abram: `http://localhost:8080/cadastro`

A tela de cadastro aparece, com o CSS.

Se clicarem em Cadastrar agora, dá erro.

Ainda não existe o POST. É o próximo passo.

Se aparecer só a palavra “cadastro”

A classe está com `@RestController`. Troquem para `@Controller`.

Passo 9 — Receber o formulário

Onde estamos

Ainda em `AcessoController.java`

```
@PostMapping("/cadastro")
public String cadastrar(Conta conta, Model model,
                        RedirectAttributes redirect) {
    if (contaRepository.findByCpf(conta.getCpf()) != null) {
        model.addAttribute("erro", "Já existe uma conta com esse CPF");
        return "cadastro";
    }

    conta.setSaldo(0);
    contaRepository.save(conta);

    redirect.addFlashAttribute("sucesso", "Conta criada! Agora é só entrar.");
    return "redirect:/cadastro";
}
```

Amanhã, quando o login existir, o último return muda.

Lendo o POST do cadastro

- `Conta conta` O Spring pega os `name` do formulário e preenche uma `Conta` nova: `nome`, `cpf`, `senha`.
- `findByCpf` Se já existe alguém com esse CPF, mostra o erro e devolve a mesma tela.
- `save` Grava no banco. Aqui acontece o `INSERT`.
- `addFlashAttribute` Um recado que sobrevive ao `redirect`. Aparece uma vez e some.
- `redirect` Depois do `POST`, sempre `redirect`.

Imports do AcessoController

- Conta e ContaRepository
- Controller — de `org.springframework.stereotype`
- Autowired, GetMapping, PostMapping
- Model — de `org.springframework.ui`
- RedirectAttributes

Se o VS Code oferecer mais de um Model, escolham o de `org.springframework.ui`.

Passo 10 — Cadastrar e conferir no banco

- 1 Reiniciem o Spring
- 2 Em /cadastro, criem uma conta com o nome de vocês
- 3 A mensagem verde aparece
- 4 No terminal, aparece um `insert into conta`
- 5 No phpMyAdmin: tabela conta, aba **Visualizar**

A linha está lá, com id 1 e saldo 0.

Tentem cadastrar o mesmo CPF de novo: a mensagem vermelha aparece.

A prova de que ficou gravado

Agora o teste que o `ArrayList` nunca passou:

- 1 Parem o Spring (Ctrl+C)
- 2 Subam de novo
- 3 Olhem o phpMyAdmin

A conta continua lá.

Podem até desligar o computador. Amanhã ela ainda vai estar lá.

Olhando pela aba SQL

No phpMyAdmin, com o banco banco aberto, aba **SQL**:

```
SELECT * FROM conta;
```

```
SELECT nome, cpf FROM conta WHERE nome LIKE 'A%';
```

O primeiro mostra todas as contas.

O segundo, só nome e CPF de quem começa com A.

Não precisa decorar. É só para ver que o dado está no banco.

Conferindo o Dia 1

XAMPP com MySQL verde

O Spring sobe sem erro

A tabela conta apareceu sozinha

/cadastro abre com o CSS

Cadastrar mostra a mensagem verde

A conta aparece no phpMyAdmin

CPF repetido mostra a mensagem vermelha

Reiniciar o Spring não apaga a conta

Erros que mais aparecem

- Communications link failure — o MySQL do XAMPP está desligado
- Unknown database 'banco' — faltou criar o banco no phpMyAdmin
- Access denied for user 'root' — a senha no application.properties está errada
- Página de erro com template might not exist — o arquivo não está em templates/, ou o nome no return está diferente
- Tabela não aparece — faltou o @Entity
- Página sem CSS — o link do CSS está diferente de /css/estilo.css

Amanhã: login, saldo, depósito, saque,
PIX e extrato.

Não apaguem o banco nem as contas.
Amanhã a gente entra com elas.

Onde paramos ontem

- O XAMPP liga o MySQL
- O Spring cria a tabela conta sozinho
- /cadastro grava uma conta nova
- Os dados ficam no banco mesmo depois de reiniciar

Antes de tudo, hoje

- 1 XAMPP: Start no Apache e no MySQL
- 2 Abram o projeto banco e subam o Spring
- 3 Confiram no phpMyAdmin: as contas de ontem estão lá?

O plano de hoje

Parte 3 — 18h30 às 20h

Tela de login, sessão, página da conta, depósito e saque.

Parte 4 — 20h20 às 22h

PIX, extrato e o teste final com pessoas do gerador.

Ao final, cada um terá um banco funcionando, com o dinheiro indo de uma conta para outra.

Passo 11 — login.html, o topo

Onde estamos

Arquivo novo templates/login.html.

Copiem o <head> do cadastro e troquem o título para Entrar.

```
<body>
  <div class="cartao">
    <p class="marca">Banco Entra21</p>
    <h1 class="titulo">Entrar</h1>

    <p class="alerta-sucesso" th:if="${sucesso}" th:text="${sucesso}"></p>
    <p class="alerta-erro" th:if="${erro}" th:text="${erro}"></p>
```

É o mesmo começo do cadastro, só muda o título.

Passo 11 — login.html, o formulário

Onde estamos

Continuação de templates/login.html

```
<form th:action="@{/login}" method="post">
  <label>CPF</label>
  <input type="text" name="cpf" maxlength="11" required>

  <label>Senha</label>
  <input type="password" name="senha" required>

  <button class="botao" type="submit">Entrar</button>
</form>

<a class="link" th:href="@{/cadastro}">Abrir uma conta</a>
</div>
</body>
</html>
```

Passo 12 — Mostrar o login

Onde estamos

AcessoController.java, acrescentar

```
@GetMapping("/")
public String inicio() {
    return "redirect:/login";
}

@GetMapping("/login")
public String telaLogin() {
    return "login";
}
```

Quem abrir `http://localhost:8080` cai direto no login.

Passo 13 — Cadastro agora leva ao login

Onde estamos

AcessoController.java, no fim do POST /cadastro

Ontem era:

```
return "redirect:/cadastro";
```

Troquem por:

```
return "redirect:/login";
```

A mensagem “Conta criada!” agora aparece na tela de login.
Por isso o login.html também tem o th:if do sucesso.

Como funciona o login

A pessoa manda CPF e senha. O servidor:

- 1 Procura a conta pelo CPF (`findByCpf`)
- 2 Se não achou, erro
- 3 Se achou, mas a senha é diferente, erro
- 4 Se deu tudo certo, guarda o id da conta na sessão
- 5 Manda para `/conta`

O erro é o mesmo nos dois casos: “CPF ou senha incorretos”.
Assim ninguém descobre quais CPFs têm conta.

Passo 14 — POST do login

Onde estamos

AcessoController.java, acrescentar

```
@PostMapping("/login")
public String entrar(@RequestParam String cpf, @RequestParam String senha,
                    Model model, HttpSession session) {
    Conta conta = contaRepository.findByCpf(cpf);

    if (conta == null || !conta.getSenha().equals(senha)) {
        model.addAttribute("erro", "CPF ou senha incorretos");
        return "login";
    }

    session.setAttribute("contaId", conta.getId());
    return "redirect:/conta";
}
```

Lendo o POST do login

`findByCpf` Procura a conta no banco. Se não achar, vem `null`.

`||` Quer dizer “ou”. Não achou a conta **ou** a senha é outra.

`equals` Compara texto. Com `String`, nunca usem `==`.

`!` Quer dizer “não”. `!equals` é “não é igual”.

`return` Devolve a mesma tela de login, agora com o erro.

A sessão por dentro

```
session.setAttribute("contaId", conta.getId());
```

É o crachá sendo entregue.

Guardamos só o **id**, com a etiqueta "contaId".

Cada navegador tem a sua sessão.

Se abrirem uma janela anônima, podem entrar com outra conta ao mesmo tempo.

Import: `jakarta.servlet.http.HttpSession`.

Onde estamos

AcessoController.java, acrescentar

```
@GetMapping("/sair")
public String sair(HttpSession session) {
    session.invalidate();
    return "redirect:/login";
}
```

invalidate rasga o crachá.

Tudo que estava na sessão é esquecido.

Uma palavra sobre a senha

Estamos guardando a senha do jeito que a pessoa digitou.
Quem abrir o phpMyAdmin consegue ler.

Isso serve só para aprender.

Em sistema de verdade:

- a senha é guardada embaralhada, sem volta (*hash*)
- o login é feito pelo **Spring Security**

Nunca usem uma senha de verdade de vocês nesse projeto.

Passo 16 — ContaController

Onde estamos

Arquivo novo controller/ContaController.java

```
@Controller
public class ContaController {

    @Autowired
    private ContaRepository contaRepository;

    public Conta contaLogada(HttpSession session) {
        Long id = (Long) session.getAttribute("contaId");
        if (id == null) {
            return null;
        }
        return contaRepository.findById(id).orElse(null);
    }
}
```

Uma função que olha o crachá e devolve a conta, ou `null` se ninguém entrou.

`getAttribute` Pega o que está na sessão com a etiqueta "contaId".

`(Long)` A sessão guarda tudo como `Object`.

O `(Long)` avisa ao Java: "isso aqui é um `Long`".

`id == null` Ninguém fez login nesse navegador.

`findById` Busca a conta no banco pelo id.

`orElse(null)` Se não achar, devolve `null`.

Por que buscar no banco de novo?

Porque o saldo pode ter mudado. Por exemplo, alguém mandou um PIX.

Passo 17 — GET /conta

Onde estamos

ContaController.java, acrescentar

```
@GetMapping("/conta")
public String telaConta(HttpSession session, Model model) {
    Conta conta = contaLogada(session);
    if (conta == null) {
        return "redirect:/login";
    }

    model.addAttribute("conta", conta);
    return "conta";
}
```

Sem crachá, volta para o login.

Com crachá, coloca a conta na bandeja e mostra a página.

Passo 18 — conta.html, o topo

Onde estamos

Arquivo novo templates/conta.html.

Copiem o <head> do cadastro e troquem o título para Minha conta.

```
<body>
  <div class="cartao cartao-larga">
    <div class="topo">
      <p class="marca">Banco Entra21</p>
      <a class="botao-secundario" th:href="@{/sair}">Sair</a>
    </div>

    <p class="ola">Olá, <strong th:text="${conta.nome}">Nome</strong></p>
```

`${conta.nome}`: a conta que o Controller pôs na bandeja,
e dela o `getNome()`.

Passo 18 — conta.html, o saldo

Onde estamos

Continuação de templates/conta.html

```
<div class="saldo">
  <p class="saldo-rotulo">Saldo disponível</p>
  <p class="saldo-valor" th:text="'R$ ' +
    ${#numbers.formatDecimal(conta.saldo, 1, 'POINT', 2, 'COMMA')}">
    R$ 0,00</p>
</div>

<p class="alerta-sucesso" th:if="${sucesso}" th:text="${sucesso}"></p>
<p class="alerta-erro" th:if="${erro}" th:text="${erro}"></p>
```

A linha comprida do saldo é explicada no próximo slide.

Formatando o dinheiro

```
${#numbers.formatDecimal(conta.saldo, 1, 'POINT', 2, 'COMMA')}
```

Sem isso, 1234.5 aparece como 1234.5.

Com isso, aparece 1.234,50.

Pedaço	Significa
<code>conta.saldo</code>	o número
<code>1</code>	pelo menos 1 dígito antes da vírgula
<code>'POINT'</code>	separa os milhares com ponto
<code>2</code>	duas casas depois da vírgula
<code>'COMMA'</code>	os centavos vêm depois de uma vírgula

Passo 19 — conta.html, o depósito

Onde estamos

Continuação de templates/conta.html

```
<div class="operacoes">
  <form class="operacao" th:action="@{/deposito}" method="post">
    <h3>Depósito</h3>
    <label>Valor</label>
    <input type="number" name="valor" step="0.01" min="0.01" required>
    <button class="botao" type="submit">Depositar</button>
  </form>
```

step="0.01" deixa digitar centavos.

min="0.01" não deixa digitar zero nem negativo.

Passo 19 — conta.html, o saque

Onde estamos

Continuação de templates/conta.html

```
<form class="operacao" th:action="@{/saque}" method="post">
  <h3>Saque</h3>
  <label>Valor</label>
  <input type="number" name="valor" step="0.01" min="0.01" required>
  <button class="botao" type="submit">Sacar</button>
</form>
</div>
</div>
</body>
</html>
```

O primeiro </div> fecha as operações.

O segundo fecha o cartão.

Testando a página da conta

- 1 Reiniciem o Spring
- 2 Abram `http://localhost:8080`: cai no login
- 3 Entrem com a conta de ontem
- 4 Aparece o nome e R\$ 0,00
- 5 Cliquem em Sair e tentem abrir `/conta` direto
- 6 Volta para o login: sem crachá, não entra

Os botões de depósito e saque ainda dão erro.
É o próximo passo.

Passo 20 — Depósito, as conferências

Onde estamos

ContaController.java, acrescentar

```
@PostMapping("/deposito")
public String depositar(@RequestParam double valor, HttpSession session,
                        RedirectAttributes redirect) {
    Conta conta = contaLogada(session);
    if (conta == null) {
        return "redirect:/login";
    }

    if (valor <= 0) {
        redirect.addFlashAttribute("erro", "Digite um valor maior que zero");
        return "redirect:/conta";
    }
}
```

O método continua no próximo slide.

Passo 20 — Depósito, somando

Onde estamos

Ainda no método depositar

```
conta.setSaldo(conta.getSaldo() + valor);  
contaRepository.save(conta);  
  
redirect.addFlashAttribute("sucesso", "Depósito feito");  
return "redirect:/conta";  
}
```

Lendo o depósito

- 1 `@RequestParam double valor` — vem do `name="valor"`
- 2 Confere o crachá
- 3 Recusa valor zero ou negativo
- 4 Soma no saldo do objeto
- 5 `save` — como a conta já tem `id`, vira `UPDATE`
- 6 Deixa o recado verde e volta para `/conta`

O `/conta` busca a conta de novo no banco.

Por isso o saldo aparece atualizado.

Passo 21 — Saque, as conferências

Onde estamos

ContaController.java, acrescentar

```
@PostMapping("/saque")
public String sacar(@RequestParam double valor, HttpSession session,
                    RedirectAttributes redirect) {
    Conta conta = contaLogada(session);
    if (conta == null) {
        return "redirect:/login";
    }

    if (valor <= 0) {
        redirect.addFlashAttribute("erro", "Digite um valor maior que zero");
        return "redirect:/conta";
    }
}
```

Até aqui é igual ao depósito.

Passo 21 — Saque, tirando do saldo

Onde estamos

Ainda no método sacar

```
if (conta.getSaldo() < valor) {  
    redirect.addFlashAttribute("erro", "Saldo insuficiente");  
    return "redirect:/conta";  
}  
  
conta.setSaldo(conta.getSaldo() - valor);  
contaRepository.save(conta);  
  
redirect.addFlashAttribute("sucesso", "Saque feito");  
return "redirect:/conta";  
}
```

O saque é o depósito com uma conferência a mais

Depósito	Saque
confere o crachá	confere o crachá
valor maior que zero	valor maior que zero
—	saldo suficiente?
saldo + valor	saldo - valor
save	save

Igual à venda da aula 06:
primeiro confere o estoque, depois tira.

Imports do ContaController

- Conta e ContaRepository
- Controller, Autowired
- GetMapping, PostMapping, RequestParam
- Model — de `org.springframework.ui`
- RedirectAttributes
- HttpSession — de `jakarta.servlet.http`

Se faltar um import, o VS Code sublinha em vermelho.
Aceitem o import que ele oferece, olhando o nome do pacote.

Conferindo a Parte 3

/ leva para o login

Senha errada mostra “CPF ou senha incorretos”

Login certo mostra o nome e o saldo

Depositar 500 mostra R\$ 500,00

Sacar mais do que tem mostra “Saldo insuficiente”

No phpMyAdmin o saldo mudou

Sair e abrir /conta volta para o login

F5 depois do depósito **não** deposita de novo

Erros que mais aparecem

- Required parameter 'valor' is not present — o name do input não é valor
- Saldo não muda — faltou o `contaRepository.save(conta)`
- Exception evaluating SpringEL no `conta.html` — o Controller não fez `model.addAttribute("conta", ...)`
- Sempre volta para o login — no `setAttribute` e no `getAttribute` a etiqueta está diferente
- Mensagem não aparece — usaram `model` em vez de `redirect.addFlashAttribute` antes de um `redirect`

Retomamos às **20h20**

Na volta: PIX entre contas e o extrato.
Deixem pelo menos duas contas cadastradas.

O que é um PIX no nosso banco

Dinheiro sai de uma conta e entra em outra.
Quem recebe é encontrado pelo CPF.

O servidor faz nesta ordem:

- 1 Confere o crachá: quem está mandando
- 2 Valor maior que zero
- 3 Acha a conta de destino pelo CPF
- 4 Não deixa mandar para si mesmo
- 5 Confere se quem manda tem saldo
- 6 Tira de um, põe no outro, grava os dois

Primeiro confere **tudo**. Só depois mexe no dinheiro.

Passo 22 — O formulário do PIX

Onde estamos

templates/conta.html, dentro da div operacoes, depois do formulário de saque

```
<form class="operacao" th:action="@{/pix}" method="post">
  <h3>PIX</h3>
  <label>CPF de quem recebe</label>
  <input type="text" name="cpfDestino" maxlength="11" required>
  <label>Valor</label>
  <input type="number" name="valor" step="0.01" min="0.01" required>
  <button class="botao" type="submit">Enviar PIX</button>
</form>
```

Dois campos: cpfDestino e valor.

O CSS já coloca os três formulários lado a lado.

Passo 23 — PIX, o começo

Onde estamos

ContaController.java, acrescentar

```
@Transactional
@PostMapping("/pix")
public String pix(@RequestParam String cpfDestino, @RequestParam double valor,
                  HttpSession session, RedirectAttributes redirect) {
    Conta origem = contaLogada(session);
    if (origem == null) {
        return "redirect:/login";
    }

    if (valor <= 0) {
        redirect.addFlashAttribute("erro", "Digite um valor maior que zero");
        return "redirect:/conta";
    }
}
```

origem é quem manda. Quem recebe vem no próximo slide.

Passo 23 — PIX, achando quem recebe

Onde estamos

Ainda no método pix

```
Conta destino = contaRepository.findByCpf(cpfDestino);  
if (destino == null) {  
    redirect.addFlashAttribute("erro", "Nenhuma conta com esse CPF");  
    return "redirect:/conta";  
}  
  
if (destino.getId().equals(origem.getId())) {  
    redirect.addFlashAttribute("erro", "Não dá para mandar PIX para você mesmo");  
    return "redirect:/conta";  
}
```

Por que equals para comparar os ids?

Long com L maiúsculo é objeto. Objeto se compara com equals.

Passo 23 — PIX, mexendo no dinheiro

Onde estamos

Ainda no método pix

```
if (origem.getSaldo() < valor) {  
    redirect.addFlashAttribute("erro", "Saldo insuficiente");  
    return "redirect:/conta";  
}  
  
origem.setSaldo(origem.getSaldo() - valor);  
destino.setSaldo(destino.getSaldo() + valor);  
contaRepository.save(origem);  
contaRepository.save(destino);  
  
redirect.addFlashAttribute("sucesso",  
    "PIX enviado para " + destino.getNome());  
return "redirect:/conta";  
}
```

Tira de um, põe no outro, grava os dois.

@Transactional: tudo ou nada

Imaginem que a luz acaba bem no meio do PIX:

- O `save(origem)` já rodou: a Ana perdeu 100
- O `save(destino)` não rodou: o Bruno não recebeu
- Os 100 reais sumiram

O `@Transactional` em cima do método diz ao banco:
“ou grava tudo que tem aqui dentro, ou não grava nada”.

Cuidado no import

O certo é `org.springframework.transaction.annotation.Transactional`.

Conferindo o PIX

Use duas contas: uma no navegador normal,
outra numa janela anônima.

PIX para um CPF que não existe: mensagem vermelha

PIX para o próprio CPF: mensagem vermelha

PIX maior que o saldo: “Saldo insuficiente”

PIX certo: o saldo cai de um lado

F5 na janela anônima: o saldo sobe do outro lado

Por que outra tabela

O saldo diz **quanto** tem.

Não diz **o que aconteceu**.

Para o extrato, cada operação vira uma linha nova:

id	conta_id	descricao	valor	data_hora
1	1	Depósito	500.50	23/09 19:02
2	1	Saque	-100.00	23/09 19:05
3	1	PIX enviado para Bruno Lima	-150.25	23/09 20:31
4	2	PIX recebido de Ana Souza	150.25	23/09 20:31

O `conta_id` diz de quem é a linha.

Um PIX gera **duas** linhas: uma em cada conta.

Saiu ou entrou?

Na coluna valor:

- Entrou dinheiro: número **positivo**
depósito, PIX recebido
- Saiu dinheiro: número **negativo**
saque, PIX enviado

Assim, no extrato, é fácil pintar:
verde para positivo, vermelho para negativo.

Passo 24 — A classe Movimentacao

Onde estamos

Arquivo novo model/Movimentacao.java

```
@Entity
public class Movimentacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String descricao;

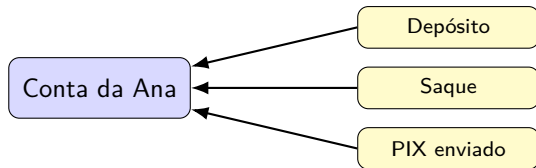
    private double valor;

    private LocalDateTime dataHora;

    @ManyToOne
    private Conta conta;
```

Construtor vazio e gets e sets dos cinco atributos.

@ManyToOne — muitas para uma



Muitas movimentações apontam para **uma** conta.

No banco, isso vira a coluna `conta_id`.

Na aula 05 a gente escrevia `int categoriaId` na mão.

Agora a gente põe a Conta inteira e o JPA guarda só o id.

`LocalDateTime` guarda dia e hora juntos.

`LocalDateTime.now()` pega o momento de agora.

No banco vira a coluna `data_hora`.

O JPA troca o `dataHora` do Java pelo `data_hora` do banco.

Imports:

- `java.time.LocalDateTime`
- `ManyToOne` e as outras anotações de `jakarta.persistence`

Passo 25 — MovimentacaoRepository

Onde estamos

Arquivo novo repository/MovimentacaoRepository.java

```
public interface MovimentacaoRepository
    extends JpaRepository<Movimentacao, Long> {

    List<Movimentacao> findByContaOrderByDataHoraDesc(Conta conta);
}
```

Pedaço	Significa
findBy	buscar onde
Conta	a conta é esta
OrderBy	em ordem de
DataHora	data e hora
Desc	da mais nova para a mais velha

Passo 26 — Uma função para registrar

Onde estamos

ContaController.java, acrescentar

```
@Autowired
private MovimentacaoRepository movimentacaoRepository;

public void registrar(Conta conta, String descricao, double valor) {
    Movimentacao mov = new Movimentacao();
    mov.setConta(conta);
    mov.setDescricao(descricao);
    mov.setValor(valor);
    mov.setDataHora(LocalDateTime.now());
    movimentacaoRepository.save(mov);
}
```

Em vez de repetir essas linhas em cada operação, chamamos registrar com os três dados.

Passo 27 — Registrar cada operação

Onde estamos

ContaController.java, logo depois de cada save

No depósito:

```
registrar(conta, "Depósito", valor);
```

No saque, com o menos na frente:

```
registrar(conta, "Saque", -valor);
```

No PIX, uma linha para cada conta:

```
registrar(origem, "PIX enviado para " + destino.getNome(), -valor);  
registrar(destino, "PIX recebido de " + origem.getNome(), valor);
```

Passo 28 — Mandar o extrato para a página

Onde estamos

ContaController.java, no GET /conta

```
model.addAttribute("conta", conta);  
model.addAttribute("movimentacoes",  
    movimentacaoRepository.findByContaOrderByDataHoraDesc(conta));  
return "conta";
```

A linha do meio é nova.

Coloca na bandeja a lista de movimentações dessa conta.

Imports novos no Controller:

Movimentacao, MovimentacaoRepository, LocalDateTime, Transactional.

Passo 29 — O extrato, o cabeçalho

Onde estamos

templates/conta.html, depois da div operacoes,
antes de fechar a div cartao

```
<h2 class="subtitulo">Extrato</h2>

<table class="extrato">
  <tr>
    <th>Data</th>
    <th>Descrição</th>
    <th>Valor</th>
  </tr>
```

Até aqui é HTML comum: o título e a primeira linha da tabela.

Passo 29 — O extrato, uma linha por movimentação

Onde estamos

Continuação, logo abaixo

```
<tr th:each="mov : ${movimentacoes}">
  <td th:text="${#temporals.format(mov.dataHora, 'dd/MM/yyyy HH:mm')}"></td>
  <td th:text="${mov.descricao}"></td>
  <td th:text="${#numbers.formatDecimal(mov.valor, 1, 'POINT', 2, 'COMMA')}"
      th:classappend="${mov.valor < 0} ? 'saida' : 'entrada'"></td>
</tr>
</table>

<p class="vazio" th:if="${movimentacoes.isEmpty()}">Nenhuma movimentação ainda.</p>
```

Lendo o extrato

`th:each` Repete o `<tr>` para cada `mov` da lista.

`#temporals.format` Escreve a data do jeito brasileiro:
23/09/2026 20:31.

`formatDecimal` O mesmo que usamos no saldo.

`th:classappend` Acrescenta uma classe do CSS:
saida (vermelho) se o valor for negativo,
entrada (verde) se não for.

`isEmpty()` Se a lista estiver vazia, mostra o aviso.

O `?` : é um `if` curto: “é negativo? então saida, senão entrada”.

Testando o extrato

- 1 Reiniciem o Spring
- 2 No terminal: `create table movimentacao`
- 3 No phpMyAdmin: a tabela nova, com a coluna `conta_id`
- 4 Entrem, depositem, saquem e mandem um PIX
- 5 Cada operação aparece no extrato, a mais nova em cima
- 6 Entrem na outra conta: o PIX recebido aparece em verde

O que foi feito antes do extrato não aparece.

Só a partir de agora cada operação é registrada.

Pessoas de mentira, CPF válido

Para testar, vamos criar pessoas que não existem.

- 1 Abram `https://www.4devs.com.br/gerador_de_pessoas`
- 2 Marquem para gerar o CPF **sem pontuação**
- 3 Cliquem em **Gerar Pessoa**
- 4 Anotem o nome e o CPF
- 5 Façam isso três vezes

O CPF gerado é válido, mas não é de ninguém.

Por isso não usamos o CPF de vocês.

Roteiro do teste

- 1 Cadastrem as três pessoas, com senhas simples
- 2 Entrem com a primeira e depositem R\$ 1.000
- 3 Mandem R\$ 300 de PIX para a segunda
- 4 Mandem R\$ 200 de PIX para a terceira
- 5 Saquem R\$ 50
- 6 Saiam e entrem com a segunda: saldo R\$ 300
- 7 Da segunda, mandem R\$ 100 para a terceira
- 8 Entrem com a terceira: saldo R\$ 300, dois PIX recebidos

Conferindo pelo phpMyAdmin

```
SELECT nome, saldo FROM conta;
```

```
SELECT * FROM movimentacao;
```

```
SELECT SUM(saldo) FROM conta;
```

O último soma o saldo de todas as contas.

Depois do roteiro, deve dar R\$ 950: os 1.000 depositados menos os 50 sacados.

PIX não cria nem some dinheiro. Só muda de lugar.

Cadastro grava no MySQL e recusa CPF repetido

Login recusa senha errada

/conta sem login volta para o login

Saldo aparece como R\$ 1.234,56

Saque e PIX recusam quando falta saldo

PIX recusa CPF que não existe e o próprio CPF

Extrato mostra tudo, o mais novo em cima, com cores

Reiniciar o Spring não perde nada

Erros que mais aparecem

- Tabela movimentacao sem a coluna conta_id — faltou o @ManyToOne
- Extrato vazio sempre — faltou chamar registrar ou faltou o `addAttribute("movimentacoes", ...)`
- Exception evaluating SpringEL: isEmpty — o GET /conta não manda movimentacoes
- Valores do PIX invertidos — o menos ficou no registrar errado
- Data com formato estranho — conferir o `#temporals.format`

O que vimos nesses dois dias

- 1 Thymeleaf: o servidor devolve a página já pronta
- 2 `th:text`, `th:if`, `th:each`, `th:action`
- 3 Formulário comum com `@RequestParam` e redirect depois do POST
- 4 MySQL no XAMPP e o phpMyAdmin para olhar dentro
- 5 `@Entity`, `Repository` e `save` no lugar do `ArrayList`
- 6 Sessão como crachá do login
- 7 `@Transactional` para o PIX ser tudo ou nada
- 8 `@ManyToOne` ligando movimentação e conta

ATM (aula 07)	Banco (aula 08)
uma conta só	várias contas, com login
saldo numa variável	saldo no MySQL
parou o Spring, zerou	parou o Spring, continua
JavaScript monta a tela	Thymeleaf monta a tela
Axios chama /api/conta	o formulário chama /deposito
sem histórico	extrato completo

Se sobrar tempo

- 1 Mostrar o CPF da pessoa embaixo do nome, na página da conta
- 2 Não deixar sacar mais de R\$ 1.000 de uma vez
- 3 Na tela de cadastro, recusar CPF que não tenha 11 caracteres

Os três usam só o que vimos hoje:

um `th:text`, um `if` no Controller, um `length()`.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21